Day 20 — Namespaces & Converters

Part A — XML Namespace

1. What Is a Namespace?

> **Namespace:** it is nothing but a **unique identifier space** for that particular field.

It exists to **avoid ambiguity between two fields** that have the same name.

2. The Problem — Two Fields with the Same Name

Scenario: you are getting **multiple records from the source**. The input payload has **Book information, CD information** or other information — and in each and every data there is a **common field called** `id`.

```
<root>
  <book>
    <id>123</id>
    <name>CPI Book</name>
  </book>
  <cd>
    <id>456</id>
    <name>movie songs</name>
  </cd>
</root>
```

Here we get the `<id>` field **two times**. **How can you differentiate which id and name belong to which?** We cannot say directly. For example, if you read `//id` (XPath) in a Content Modifier — **which id will it read?** We cannot define it. **So, when we work on multiple payloads, to bring a unique identity to the particular field, we use a namespace.**

3. The Solution — Namespaces

```
<root>
  <ns0:books xmlns:ns0="https://books.com">
    <ns0:id>123</ns0:id>
    <ns0:name>CPI Book</ns0:name>
  </ns0:books>
  <ns1:cd xmlns:ns1="https://cds.com">
    <ns1:id>123</ns1:id>
    <ns1:name>Movies Songs</ns1:name>
  </ns1:cd>
</root>
```

The namespace brings uniqueness for that particular field — `ns0:id` (from books.com) and `ns1:id` (from cds.com) are now two different fields.

Part	What It Is
`xmlns:ns0="https://books.com"`	The **namespace declaration**
`https://books.com`	The **namespace itself** — the unique identifier (it is just a name; it doesn't have to be a live website)
`ns0`	The **prefix** — a short nickname for the namespace, used in the tags

> **Small fixes and nuance:**
>
> - **Prefix spelling:** in your sketch the prefix looks like "nso" (letter O). The usual prefix is **ns0** (letter N, S, **zero**) — your own note "zero" above it says the same. A prefix is just a nickname, so what matters is that the opening tag, closing tag and declaration all use the same one.

- **Absolute XPath can also tell them apart:** in the first (non-namespaced) payload, `/root/book/id` and `/root/cd/id` are different paths. The ambiguity is really with the **relative** path `//id`. Namespaces matter most when fields from **different systems or schemas** are combined, and because many SAP/SOAP/IDoc payloads and target schemas **require** them.
- **Reading a namespaced field:** the XPath must use the prefix (e.g., `/root/ns0:books/ns0:id`), and the prefix must be declared in the iFlow's **Runtime Configuration → Namespace Mapping** — otherwise CPI can't find the element.

Link to Day 19 — "Use Namespace Mapping" in XML → JSON

This is exactly what the converter's **Use Namespace Mapping** option (Day 19) deals with. If it is **checked**, the namespace prefix is carried into the JSON using the **JSON prefix separator** (e.g., `books:id`). If it is **unchecked** — **most of the time** — the prefixes are dropped, so same-named fields from different namespaces would look identical in the JSON.

Part B — Converters

4. JSON to XML Converter

#	Option	Value Shown in Class
①	**Use Namespace Mapping**	☐ (unchecked)
②	**JSON Prefix Separator**	**Colon (:)**
③	**Add XML Root Element**	☑ (checked) — Name: **root**

Why "Add XML Root Element" is ticked: an XML document must have **exactly one root** (Day 19), but JSON — especially an array of records — has none. Ticking this option wraps the converted output in a root element you name (here, `root`), so the XML is valid.

5. Working with Files — CSV and Text

CPI also works with **plain files** — a **CSV file** (Comma Separated Values) or **text files** as well **(remember)**.

Real-time scenario: you work with files, and the real work will be **CSV ↔ XML**. For example, with an **SFTP server** they will ask you to send a **(text) file**, not an XML or JSON file — **we can get any requirement.** So converting an XML to CSV needs the **XML to CSV Converter**.

(This ties directly to the SFTP adapter in your curriculum — SFTP file in/out, with the Converter in the middle.)

6. XML to CSV Converter

Basic Elements

#	Option	Meaning
①	**Path to Source Element in XML**	**Which XPath I need to work on** — the repeating element; each occurrence becomes one CSV row
②	**Field Separator in CSV**	Comma (,) — or another character such as semicolon; you can also type your own
③	**CSV Header**	**XML based** ✓ (header names come from the XML element names) *or* **XSD annotation based** (header names come from the documentation tag in an XSD you supply)

Parent Element

Option	Meaning
Include Parent Element ☐	Also bring fields of the parent (outer) element into the CSV
Include Attribute Values ☐	Also include XML **attribute** values (e.g., `<root category="Notebooks">`)

Worked Example from Class — Why Parent Element Matters

```
<employees>
  <status>Active Employees</status>
  <employee> ... fields ... </employee>
</employees>
```

If I am sending employee details, there may be **other elements outside the repeating `employee` block** — like the **status** tag, which sits in the root. **If you also want the status value, what will you do?** → tick **Include Parent Element**. Whenever you want the **parent element node's field value** as well in the CSV, use the XML to CSV Converter with this option, based on the requirement.

> **Added detail (verified in SAP's guide):** when you tick Include Parent Element, you also fill **Path to Parent Element** (for this example `/employees`), otherwise CPI doesn't know which parent to read. A community blog also reports that unticking the box does *not* switch it off while a parent path is still filled in — clear the path first.

Limitation — Simple Structures Only

> **XML ↔ CSV only works with simple structures:** you have **one node, and in that node you have all the fields** (e.g., `<contact><test>…</test>…</contact>`). A CSV is flat rows and columns — it can't hold deep, nested hierarchies.

For nested XML, first flatten it with a Message Mapping or XSLT (Day 9), then convert. SAP also notes the converter doesn't support XML values containing newline characters.

7. CSV to XML Converter

#	Option	Meaning
①	**XML Schema (XSD)**	Choose **Select** and pick the XSD (uploaded into the iFlow's resources)
②	**Path to Target Element in XSD**	**Which XPath you want to generate XML for** — the repeating element (e.g., `Books/Book`)
③	**Record Marker in CSV**	Optional filter: only rows whose **first value** equals the marker are converted; if blank, all rows are converted
④	**Field Separator in CSV**	Comma (,) etc.
⑤	**Exclude First Line Header** ☑	Tick it if the first CSV line is a header row, so it isn't converted as data

Why Does CSV → XML Need an XSD (and XML → CSV Doesn't)?

- **XSD → defines how the fields look: how many times each field can occur, and so on.**
- **XML → CSV:** the XML payload is in a **standard format** — the converter can **directly identify each element as a field** and treat it as a new column. **No XSD needed** (only for the optional XSD-annotation header).
- **CSV → XML:** a CSV is just text with separators — it has **no structure**. **I need the XML format in which I must send the data**, so **an XML Schema is a prerequisite**. **Without an XSD you cannot do CSV → XML.**

Steps

1. **Get the XSD. In real time, the source people must provide the XSD file to you** — no one gives you the XML data; they give you the **XSD**. **If it is a SOAP service, they share the WSDL file.** (For practice, you can search "XML to XSD converter" online and generate an XSD from a sample XML.)
2. **Import the XSD** into the integration flow (iFlow Resources).
3. **Configure the converter:** select the XSD, set the Path to Target Element in XSD, the field separator, and Exclude First Line Header.

> **Note on your step numbers:** your notes mention "import the XSD" under both Step ① and Step ②, so I merged them into the three steps above.

> **Caution on generated XSDs:** an XSD generated from *one sample* XML guesses the rules (how often fields repeat, data types), so review it before using it. Also, **CSV columns must match the XSD** — SAP notes that if there's no header row (untick Exclude First Line Header) the values must be in the **same order** as the XSD elements, and a mismatch can raise an "XSD schema is incompatible with CSV payload" error.

Illustrative Example

CSV file:

```
id,name
123,CPI Book
456,Movie Songs
```

Settings: XSD defining `Books/Book` with `id` and `name` · Path to Target Element = `Books/Book` · Separator = comma · Exclude First Line Header ☑ → one `<Book>` element per CSV row.

8. Summary — Converters at a Glance

Converter	Needs a Schema?	Key Options
XML → JSON	No	Suppress root, namespace mapping, prefix separator, encoding, streaming (Day 19)
JSON → XML	No	Namespace mapping, prefix separator, **Add XML Root Element**
XML → CSV	Only for XSD-annotation headers	Path to source element, separator, header type, include parent / attributes
CSV → XML	**Yes — XSD required**	XSD, path to target element, record marker, separator, exclude header

9. Quick Interview Answers

Question	Short Answer
What is a namespace and why use it?	A unique identifier that qualifies element names so same-named fields from different sources don't clash
Why does CSV → XML need an XSD?	CSV has no structure; the XSD defines the target XML (fields, repetition)
What do you get from the source team for CSV → XML?	The XSD (or the WSDL for a SOAP service)
How do you include a field that sits outside the repeating element in XML → CSV?	Tick Include Parent Element and set the Path to Parent Element
Can XML → CSV handle nested XML?	No — simple, flat structures only; flatten first
Why "Add XML Root Element" in JSON → XML?	XML needs one root; JSON (e.g., an array) may not have one